

Advanced Python Day 7

Mini Project: Library Management System

Real Life Example: Library

- **Library contains:**

📖 Books, 👤 Students, 🧑 Librarian

- **Students can:**

Borrow books, Return books

- **Library must:**

Track available books, Prevent borrowing unavailable books



Real Life → Python Mapping

Real Life Concept	Python OOP Model
Library	Class
Books list	Attribute
Borrow / Return book	Method
Student	Object

💡 OOP models real-world systems.

Step 1: Create Library Class

Blueprint Design

Create the fundamental blueprint of a library system.

 Define the class.

 `books` = list of available books initialized at creation.

```
class Library:
    def __init__(self, books):
        self.books = books
```


Step 2: Show Available Books

Viewing Inventory

When a student asks: *"Which books are available?"*

We iterate through the `self.books` list and display each item to the user.

```
def show_books(self):  
    print("Available Books:")  
    for book  
in self.books: print(book)
```


Step 3: Borrow Book

Handling Transactions

Library must check if the requested book actually exists.

- ✓ If available, remove from list.
- ✗ If unavailable, show error message.

```
def borrow_book(self, book_name):  
    if book_name in self.books:  
        print("You borrowed:", book_name)  
        self.books.remove(book_name)    else:  
        print("Book not available")
```


Step 4: Return Book

Restoring Inventory

When a student is finished and returns a book.

The system appends the book back to the primary `self.books` list, making it available for others.

```
def return_book(self, book_name):  
    self.books.append(book_name)  
    print("Book returned:", book_name)
```


Complete Library System

```
class Library:
    def __init__(self, books):
        self.books = books
    def show_books(self):
        print("Available Books:")
        for book in self.books:
            print(book)
```

```
def borrow_book(self, book_name):
    if book_name in self.books:
        print("You borrowed:", book_name)
        self.books.remove(book_name)
    else:
        print("Book not available")
def return_book(self, book_name):
    self.books.append(book_name)
    print("Book returned:", book_name)
```


Running the Program

Execution Code

```
lib = Library(["Python",  
"AI", "Data"]) lib.show_books()  
lib.borrow_book("Python")  
lib.return_book("Python")
```

Console Output

```
Available Books: Python AI  
Data You borrowed: Python Book  
returned: Python
```


What Just Happened?

You created a real system model. Real systems work the exact same way across different industries:

Company (Context)	System Object
 Netflix	Movies
 Amazon	Products
 Bank	Accounts
 Library	Books

Think Before Running

Double Execution

```
lib = Library(["Python",  
"Java"]) lib.borrow_book("Python")  
lib.borrow_book("Python")
```

? Question:

Why does the second borrow fail?

Because the book was already **removed** from the list during the first transaction. The state of the object changed!

Practice Task 1: Video Game Inventory

 Class: **Inventory**

 Attribute: `items` list

 Method: `add_item()`

 Method: `remove_item()`



Practice Task 2: School Attendance

 Class: **Classroom**

 Attribute: `students` list

 Method: `mark_present()`

 Method: `show_students()`



Practice Task 3: **Movie Ticket System**

 Class: **Cinema**

 Attribute: `seats` available

 Method: `book_seat()`

 Method: `cancel_seat()`



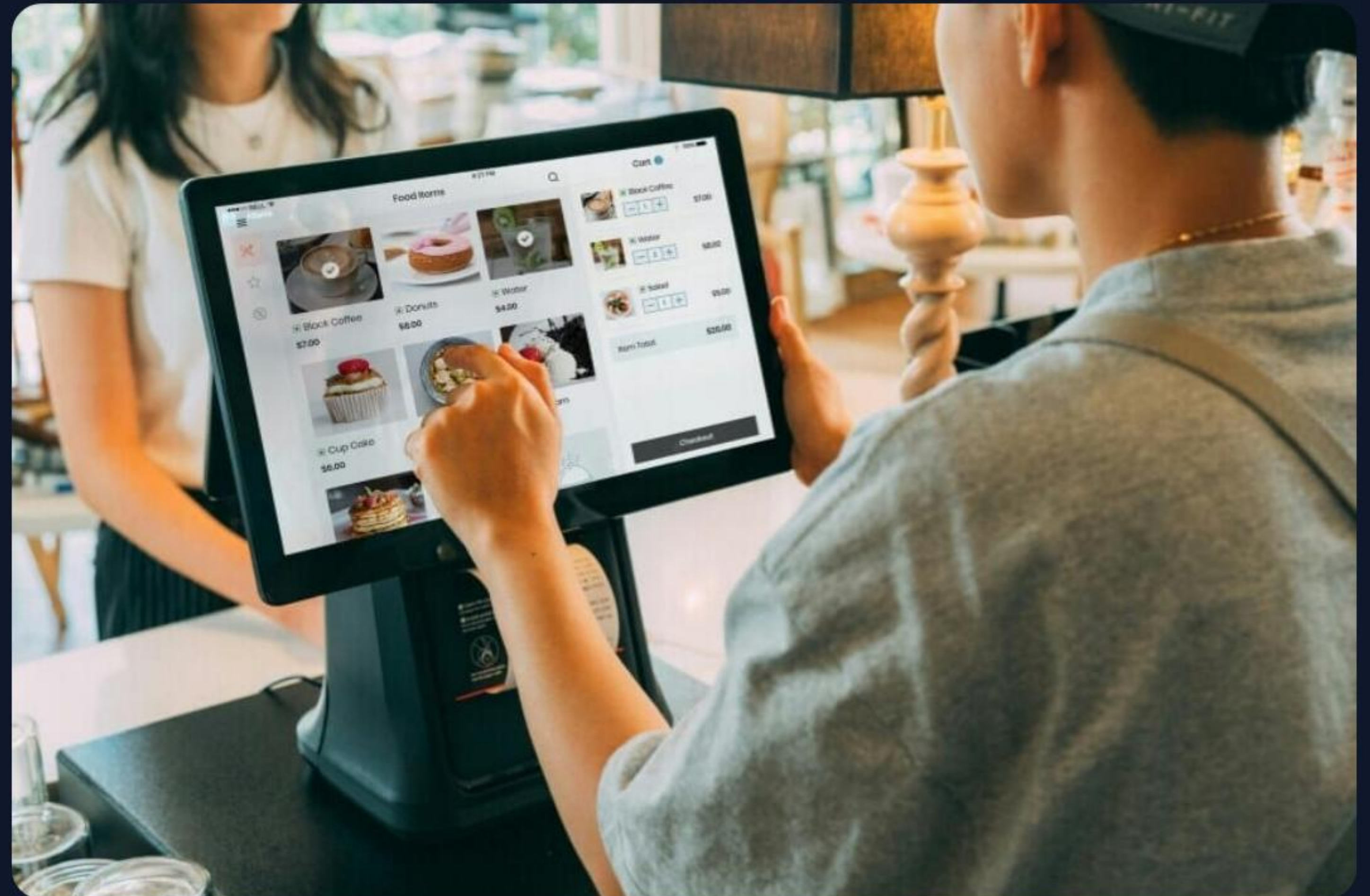
Practice Task 4: Restaurant System

 Class: Restaurant

 Attribute: menu list

 Method: order_food()

 Method: show_menu()



Developer Insight

Real applications are vast collections of interacting classes.



Uber

Driver, Rider, Trip



Instagram

User, Post, Comment



Amazon

Product, Cart, Order

Large systems = many small classes working together.

Day 7 Summary

Today you successfully:

- ✓ Modeled a real system from scratch
- ✓ Used classes and methods together cohesively
- ✓ Managed state changes using objects
- ✓ Built your first functional OOP mini project

You are now thinking like a developer! 🚀

Image Sources



https://dlwci53n4f20tv.cloudfront.net/thumbnails/_1200x630_crop_center-center_82_none/Thumb-Library-Design-Western-Kentucky-University-Commons-at-Helm-Library-Photos-by-Nick-McGinn.jpg?mtime=1769437468

Source: iida.org



https://png.pngtree.com/background/20250207/original/pngtree-neon-gaming-vibes-retro-game-controller-glow-picture-image_15950772.jpg

Source: [pngtree.com](https://png.pngtree.com)



<https://www.smarttech.com/-/media/project/smart/www/education/interactive-displays/teacher-students-smart-board-interactive-learning.jpg?h=800&iar=0&w=1200&rev=a9b235a967fd491fbe8a45f7d84d0cbe&hash=F385F00E610B52E21A74AAE475F63384>

Source: www.smarttech.com



https://static.vecteezy.com/system/resources/thumbnails/068/840/226/small_2x/empty-red-cinema-seats-facing-glowing-screen-in-dark-theater-with-ambient-lighting-photo.jpg

Source: www.vecteezy.com



https://media.superiorseating.com/Magento_Live_Site/wysiwyg/articles/restaurant-worker-processing-transaction-on-pos-system.jpg

Source: www.superiorseating.com